

Step 5 — Write fail-to-pass tests

Now that `[sol]` is committed, add the tests. Start from the tests that came with the original PR; include all of them, then add any extra cases you see fit.

Test requirements

Every test must:

- Fail on the base commit¹ (main before your solution).
- Pass after your `[sol]` commit².

¹. This proves the test targets the actual bug or missing feature

². This proves your solution fixes it

If a test passes both before and after, it's not a fail-to-pass test — update or replace it.

Undo test removal and commit

In Step 4 you removed test changes before committing. Undo that removal, then:

```
git commit -am "[f2p] fail to pass tests"
git push origin golden-solution
```

[Copy](#)

Verify your branch state

Run `git log --oneline` to confirm your branch has exactly two commits in this order:

```
$ git log --oneline

a1b2c3d [f2p] fail to pass tests
e4f5g6h [sol] golden solution
```

[Copy](#)

Check your work

Your branch has exactly two commits: `[sol]` feature code, then `[f2p]` test code. No merge commits, no extra commits. Next: fill in the metadata file and validate (Step 6).

F2P scope — three floors

Must-have (mandatory floor):

- Include every existing-solution test that is a valid F2P candidate — i.e., it fails on base, passes on `[sol]`, and runs successfully inside the Docker environment.

- Each retained test must fail on base and pass on [sol] . Verify this before committing.

Exclude (with a one-line note in the PR description):

- Tests that pass on base (not actually F2P — system-generated tests sometimes fall into this category).
- Tests that can't run inside Docker (live tests, external network dependency, missing host services).
- Tests that depend on environment that isn't reproducible in the container.

Do not omit valid F2P tests just because they look redundant — each one may catch a different model failure mode.

Should-have (coverage floor):

- Every distinct behavior named in `problem_statement` maps to at least one F2P test. Walk through the prompt sentence by sentence and check.
- At least one boundary case the prompt mentions explicitly (empty input, max size, edge of valid range).

Recommended (robustness floor):

- At least one negative test — an input the fix should NOT change.
- At least one regression test if the bug class is common (off-by-one, null handling, encoding).

Keep redundant-looking valid tests

If a test fails on base and passes on [sol] and runs in Docker, keep it — even if you think it's redundant. Different assertions catch different model failure modes. Only drop a test if it's structurally not a valid F2P (passes on base, or can't run in Docker).

For a worked illustration of the three floors, see the datasynth deep-merge example (6 test cases) on the [Walkthrough](#) page.

Tests must verify behavior, not existence

An F2P test that fails on base only because the imported symbol doesn't exist there is a **weak test**. The agent can pass it by introducing any symbol with that name — the test doesn't constrain behavior.

Behavioral by default. F2P tests should use the existing public entry points on `main` wherever those entry points can observe the behavior the task adds. New internal

helpers, private types, and implementation-only symbols should NOT be imported by F2P tests just because they exist in your solution — that forces the model into your specific implementation. The only time it's appropriate for F2P to import a net-new symbol is when that symbol is itself the public contract being added (a new public API, a new exported interface). If you can route the test through something that already exists on `main`, do that.



Behavioral

```
def test_copy_isolates_tags():
    a = TaggedLogger()
    a.tag("user", "alice")
    b = a.copy()
    b.tag("user", "bob")
    assert a.get_tag("user") == "alice"
    assert b.get_tag("user") == "bob"
```

Forces the agent to actually implement isolation.



Weak — symbol-existence only

```
def test_copyable_exists():
    from misk.logging.api import Copyable
    assert Copyable is not None
    assert hasattr(TaggedLogger, 'copy')
```

Passes by creating any class named `Copyable` with any `copy()`. Doesn't verify the copy actually isolates state.

Acceptable exception

A test that imports a new symbol is acceptable IF the same test also asserts non-trivial behavior of that symbol. A pure-import smoke test alone is not.

Don't go beyond what was asked

Every assertion your F2P set makes must derive from a sentence in `problem_statement` (or `problem_statement_variant`). If a test asserts something neither statement implies, either add that requirement to the prompt or drop the assertion.



Prompt expanded to match F2P

`problem_statement`: "Fix `merge_configs` so nested dicts deep-merge. Empty overrides sho

F2P set: same 4 assertions as before, now each derives from a prompt sentence. Behavior measured matches behavior described.

**F2P inflated beyond the prompt**

problem_statement: "Fix merge_configs so nested dicts deep-merge."

F2P set:

- ✓ test_deep_merge_nested_dicts (in prompt)
- X test_empty_override_is_no_op (not in prompt)
- X test_list_vs_dict_collision_errors (not in prompt)
- X test_max_recursion_depth_is_10 (not in prompt)

Agent fails on hidden requirements — not because the task is hard.

[Previous](#)[Next](#)